Studio Nx

백엔드 192 — GET /tags

2026-10-16

Backend(FastAPI/Python) 192 — `GET /tags`

항목	내용
파일	`studionx/backend/app/api/v1/tags.py` (신설), `app/services/tag_service.py` (신설), `app/schemas/tag.py` (신설), `app/api/v1/router.py` (`/tags` 등록), `app/schemas/sample.py` (TagItem 을 tag.py 에서 가져오도록 정리)
작업	S1-07 샘플 라이브러리: 태그 목록 (FR-224·225)
작성	Claude (AI), 2026-10-15, 사용자 검토
이전 · 다음	191 `DELETE /samples/{id}` · 193 (예정) — S1-07 백엔드 마무리 점검

1. 하는 일

태그 목록을 돌려줍니다. `GET /api/v1/tags` 입니다. 라이브러리 패널의 태그 필터를 채울 때 씁니다(FR-224).

```
{"items": [
  {"tag_id": 1, "name": "Hip-Hop", "category": "GENRE"},
  {"tag_id": 5, "name": "Kick", "category": "INSTRUMENT"},
  {"tag_id": 9, "name": "Chill", "category": "MOOD"}
]}
```

경우	결과
로그인함	200 + `{ items: [...] }`
로그인 안 함	401

정렬은 **분류 순(GENRE → INSTRUMENT → MOOD → ETC), 같은 분류 안에서는 이름순**입니다.

2. 언제 쓰나

```
var res = await fetch("/api/v1/tags");
var tags = (await res.json()).items;   // category 로 나눠 필터를 그린다
```

필터에서 고른 태그의 `tag_id` 를 그대로 `GET /samples?tag=`(186)와 `PUT /samples/{id}/tags`(190)에 보냅니다.

3. 코드

스키마 (`app/schemas/tag.py`)

```
class TagItem(BaseModel):
    tag_id: int
    name: str
    category: str


class TagListResponse(BaseModel):
    items: list[TagItem]
```

서비스 (`app/services/tag_service.py`)

```
def list_tags(db: Session) -> TagListResponse:
    rows = db.scalars(select(Tag).order_by(Tag.category, Tag.name)).all()
    return TagListResponse(
        items=[
            TagItem(tag_id=tag.tag_id, name=tag.name, category=tag.category)
            for tag in rows
        ]
    )
```

라우터 (`app/api/v1/tags.py`)

```
router = APIRouter()


@router.get("", response_model=TagListResponse)
def list_tags(user: CurrentUser, db: Db) -> TagListResponse:
    return tag_service.list_tags(db)
```

`router.py`에 `api_router.include_router(tags.router, prefix="/tags", tags=["tags"])`로 등록합니다.

문법 풀이

식	뜻
`select(Tag).order_by(Tag.category, Tag.name)`	분류 먼저, 그 안에서 이름순으로 정렬합니다
`db.scalars(...).all()`	그 조건에 맞는 Tag 행을 모두 가져옵니다
`TagItem(tag_id=…, name=…, category=…)`	ORM 행에서 필요한 칸만 골라 응답 항목으로 옮깁니다
`api_router.include_router(..., prefix="/tags")`	`/api/v1/tags` 아래에 이 라우터를 붙입니다

4. 흐름

1. 로그인을 확인합니다 (401).
2. 분류 순, 이름순으로 태그를 모두 읽습니다.
3. 필요한 칸만 골라 `{ items: [...] }`로 돌려줍니다.

5. 왜 이렇게

선택	이유
평평한 목록 + `category`	분류별로 묶어 주지 않고 각 항목에 `category`를 담습니다. 화면이 나누어 그리면 되고, 태그가 늘어도 응답 모양이 바뀌지 않습니다 (사용자 결정 2026-10-15)
분류 순 정렬	MySQL `tags.category`는 ENUM이라 정의된 차례(GENRE → INSTRUMENT → MOOD → ETC)로 정렬됩니다. 화면이 따로 정렬하지 않아도 됩니다
읽기만	태그는 기본 제공(is_system) 목록입니다. 만들기·고치기는 화면 기획이 정해지면 추가합니다
`schemas/tag.py` 분리	태그는 샘플과 다른 자원이고, 한 줄 모양(TagItem)을 샘플 응답과 함께 씁니다. 중복 정의를 피하려고 tag.py에 두고 sample.py 가 가져다 씁니다

선택	이유
`/tags` 별도 라우터	태그는 샘플에 매이지 않은 목록입니다. `/samples` 아래 두지 않고 `/tags`로 둡니다

6. 시험 (`tests/test_tag_routes.py` 신설, 4개, 전체 986개 통과)

시험	확인한 것
라우트 등록	OpenAPI 에 `GET /api/v1/tags` 가 있음
목록	200, 각 항목 칸(`tag_id`·`name`·`category`), 분류 순 · 이름 순
빈 목록	태그가 없어도 200, `items` 가 목록
로그인 필요	토큰 없이 401